Programmierung wiederholen

Sicherungsblatt zu Aufgabe 6.0 · Informatik 11 · KI

1. Klasse und Objekte

Hund

```
private int alter
private String name
```

```
public void zeigeDaten()
public void belle()
public int gibAlter()
public void setzeAlter(int neuesAlter)
public void setzeDaten(int neuesAlter, String neuerName)
```

Aufruf: `petersHund.setzeAlter(6);`

Vor dem Aufruf

`petersHund: Hund`

```
alter = 5
name = "Wuffti"
```

`inasHund: Hund`

```
alter = 8
name = "Schnuffi"
```

Nach dem Aufruf

`petersHund: Hund`

```
alter = 6
name = "Wuffti"
```

`inasHund: Hund`

```
alter = 8
name = "Schnuffi"
```

Die Klasse legt Attribute und Methoden fest. Jedes Objekt besitzt eigene Attributwerte; der Aufruf verändert nur Wuffti.

2. Methoden lesen und aufrufen

Allgemeiner Methodenkopf

```
public Rueckgabetyp methodenname(Typ parametername)
```

Beispiel aus Hund.java

```
public void setzeAlter(int neuesAlter)
```

`void`: kein Rückgabewert · `int`: Parametertyp · `neuesAlter`: Parametername.

Punktnotation: Aufruf an einem bestimmten Objekt

```
objektvariable.methodenname(argumente);
petersHund.setzeAlter(6);
```

6 ist das **Argument**; sein Wert wird an `neuesAlter` übergeben. Typ, Anzahl und Reihenfolge müssen passen. Ohne Argumente bleiben die Klammern leer.

Programmierung wiederholen

Sicherungsblatt zu Aufgabe 6.0 · Informatik 11 · KI

3. Getter und Setter

Getter: Attributwert zurückgeben

Allgemeine Form für ein Attribut

```
public Typ gibAttribut() {
    return attribut;
}
```

Beispiel aus Hund.java

```
public int gibAlter() {
    return alter;
}
```

`int`: Rückgabetyp · `()`: kein Parameter. `return` gibt den Wert zurück.

Setter: Attributwert verändern

Allgemeine Form für ein Attribut

```
public void setzeAttribut(Typ wert) {
    attribut = wert;
}
```

Beispiel aus Hund.java

```
public void setzeAlter(int neuesAlter) {
    alter = neuesAlter;
}
```

Der übergebene Wert wird gespeichert. `void` bedeutet: kein Rückgabewert.

```
petersHund.setzeAlter(7);
println(petersHund.gibAlter());  // 7
println(inasHund.gibAlter());    // 8
```

Ein Getter gibt einen Wert zurück; erst `println(...)` zeigt ihn an. Die Attribute sind `private`: Im Hauptprogramm erfolgt der Zugriff über zugängliche Methoden.

4. Zwei Parameter und ein Aufruf ohne Argumente

```
public void setzeDaten(int neuesAlter, String neuerName)
inasHund.setzeDaten(9, "Flocke");
inasHund.zeigeDaten();
```

Erst Alter (`int`), dann Name (`String`). Danach zeigt `zeigeDaten()` Flocke mit Alter 9.